COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	07/10/2026
NAME:	Seth Carlo T. Miraflor Kian Joros C. Rebagos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

What procedures did you perform?

5. Procedure

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

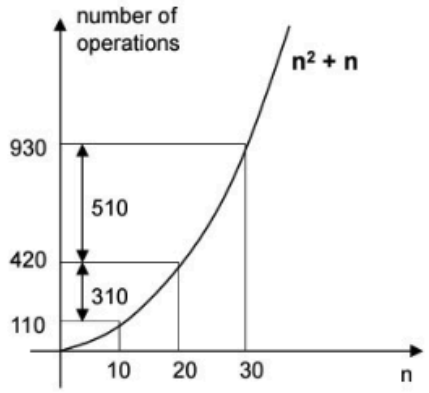
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

- **Analysis: Based on the graph, the number of operations increases as the size of the array increases. When n is 10, there is 110 operations. When n is 20, there is 420 operations. When n is 30, there is 930 operations. So this shows that the number of operations increases faster as n gets bigger and the formula is n^2 + n, so the algorithm has a quadratic growth, So the worst-case time complexity is O(n^2).**

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No
2	3	Yes

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

```
Time taken to search = 45 microseconds
Student (836 118) needs vaccination.
Time taken to search = 37 microseconds
Student (836 118) needs vaccination.
Time taken to search = 53 microseconds
Student (836 118) needs vaccination.
```

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the `Student` class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.
3. Use a **binary search** to see if the student is present on the list.
4. If the student isn't present in the list, your function should return `true` since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false.

Step 0: Begin by including the following headers:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
```



```
#include <numeric>
```

Step 1: Define the STUDENT class as follows:

```
class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;

    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
        }

        // Getters
        auto get_name() { return name; }
        auto is_vaccinated() { return vaccinated; }
```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.

```
        // Two people are same if they have the same name
        bool operator ==(const Student& p) const {
            return this->name == p.name;
        }

        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
            return this->name < p.name;
        }

        bool operator> (const Student& p) const {
            return this->name > p.name;
        }
};
```

Step 3: The following function lets us generate a student from random data:

```
auto generate_random_Student(int max) {
```

```
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

For the next important function, we will implement the use of **std::chrono for experimental analysis.**

The following code is used to run and test the output of our implementation.

```
void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point                begin                =
std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "does not need vaccination." << std::endl;
}
```

5. The following function implements our logic for whether a vaccination is needed:

```
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true)
```

```
        {
            auto range_length = std::distance(first, last);
            auto mid_element_index = std::floor(range_length / 2);
            auto mid_element = *(first + mid_element_index);

            // Return true if the Student is found in the sequence and
            // he/she's not vaccinated
            if (mid_element == P && mid_element.is_vaccinated() == false)
                return true;
                else if (mid_element == P && mid_element.is_vaccinated() == true)
                return false;
            else if (mid_element > P)
                std::advance(last, -mid_element_index);
            if (mid_element < P)
                std::advance(first, mid_element_index);

            // Student not found in the sequence and therefore should be vaccinated
            if (range_length == 1)
                return true;
        }
    }
```

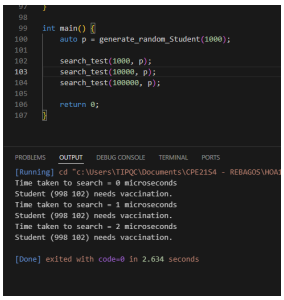
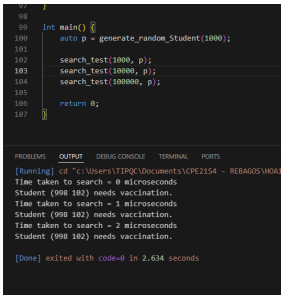
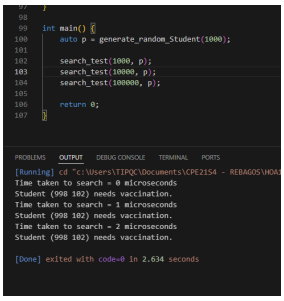

6. Finally the driver code is implemented as follows:

```
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Fill out the table below and include in section 6 table 11-1.

Input Size (N)	Execution Speed	Screenshot	Observation(s)

Provide your own analysis of the outcome listed in this table.

Input Size (N)	Execution Speed	Screenshot	Observation
1,000	0 microsecond		For N = 1,000: The search completes almost instantly (0 us) because it only takes around 10 comparison steps.
10,000	1 microsecond		For N = 10,000: The execution time increases by just (1 us) despite a 10x larger dataset, taking around 14 steps.
100,000	2 microseconds		For N = 100,000: The lookup time stays extremely fast at (2 us) for 100,000 items, taking only around 17 steps.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

7. Supplementary Activity

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.


```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Problem 1:

Theoretical Analysis

The program checks the elements of the array to see if there are duplicates. It compares many elements with each other. In the worst case it checks almost all pairs of elements. Because of this, the time complexity is O(n^2).

Experimental Analysis

When we test the program with different array sizes, the running time increases as the number of elements increases a bigger array needs more comparisons so it takes more time

Analysis and Comparison

The theoretical result is O(n^2), and the experimental result also shows that the program gets slower as

the array gets bigger. This means the experimental result is similar to the theoretical result.

Problem 2:
Theoretical Analysis
the rpower algorithm decreases n by 1 each time until it reaches 0. Because of this, its time complexity is O(n). The brpower algorithm divides n by 2 each time. This makes it faster, and its time complexity is O(log n).

Experimental Analysis
So When we test both algorithms using different values of n, rpower takes more time as n becomes bigger. The brpower algorithm takes less time because it reduces n by half each time.

Analysis and Comparison
The theoretical analysis shows that rpower is O(n), while brpower is O(log n) so experimental results also show that brpower is faster, specially for bigger values of n. So brpower is more effective than rpower.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.